Take-Home Assignment – Python Backend Engineer

1. Introduction

1.1 About This Assignment

We are looking for an experienced Python Backend Engineer who can design and implement production-grade services using clean architectural principles.

This assignment evaluates your skills in:

- Software Architecture – Designing maintainable, testable systems
- Domain Modeling – Implementing rich domain logic
- FastAPI – Building production-ready APIs
- PostgreSQL – Schema design and relational modeling
- Testing – Writing meaningful unit and integration tests
- Code Quality – Clean, idiomatic Python

1.2 Time Expectation

This assignment is designed to take **3–4 hours**.

Focus on:

- Correct architecture
- Clean code (Including proper code formatting, PEP-8 best practices)
- Proper data validation
- Testing

If you don’t complete everything, document what is done and what remains.

1.3 Required Technology Stack

You must use:

- Python 3.11+
- FastAPI
- PostgreSQL
- SQLAlchemy (ORM)
- Pydantic (request/response validation)
- Alembic or any database migration tool you're familiar with
- Pytest (testing)
- Dependency management (poetry or pip-tools)

Optional but encouraged:

- project is launchable via Docker / docker-compose

- Separate testing environment (testcontainers, PostgreSQL Test DB)
- Poetry or pip-tools for dependency management

---

2. The Assignment: Financial Ledger API

2.1 Business Context

You are tasked with building a simplified **Financial Ledger System** that follows **double-entry bookkeeping principles**.

2.1.1 Core Concepts

Account

Represents a financial account such as:

- Cash
- Revenue
- Expenses
- Liability

Each account has:

- `id` (UUID recommended)
- `name` (unique)
- `type` (ASSET, LIABILITY, REVENUE, EXPENSE)

⚠️ The balance should be calculated from transactions, not stored as a mutable field.

Transaction

A financial transaction affecting one or more accounts. Each transaction has:

- `id`
- `timestamp`
- `description`
- A collection of transaction entries

Transaction Entry

Represents a debit or credit inside a transaction.

Each entry includes:

- `account_id`
- `type` (DEBIT or CREDIT)
- `amount` (positive decimal)

2.1.2 Double-Entry Bookkeeping Rule

Critical Business Rule:

> For every transaction, total debits must equal total credits.

Example:

Purchase supplies for $100 cash:

- DEBIT: Supplies +100
- CREDIT: Cash -100

Total debits = Total credits = 100

This must be enforced at the business logic layer.

2.2 Required Features

2.2.1 Feature 1: Account Management

Create Account

> `POST /api/accounts`

Request:

```
{
"name": "Cash",
"type": "ASSET"
}
```

Response:

- 201 Created
- Account details including computed balance (initially 0)

Get All Accounts

`GET /api/accounts`

Return list of accounts with calculated balances.

Get Account by ID

> `GET /api/accounts/{id}`

Return:

- Account details
- Current calculated balance

2.2.2 Feature 2: Transaction Processing

Create Transaction

> `POST /api/transactions`

```
{
  "description": "Purchase office supplies",
  "date": "2024-01-15T10:30:00",
  "entries": [
    {
      "accountId": "uuid",
      "type": "DEBIT",
      "amount": 100.00
    },
    {
      "accountId": "uuid",
      "type": "CREDIT",
      "amount": 100.00
    }
  ]
}
```

Must validate:

- Transaction has at least 2 entries
- At least one DEBIT and one CREDIT
- All accounts exist
- Amounts are positive
- Total debits == total credits

Return 201 Created or validation error.

Get Transaction by ID

`GET /api/transactions/{id}`

Return transaction with entries.

Get Transactions for Account

`GET /api/accounts/{id}/transactions`

Return all transactions affecting that account.

2.2.3 Feature 3: Account Balance Calculation

Balance rules: Each type of account has different behaviour when creating transactions. Some accounts' balances are increased with CREDIT entries, and some are increased with DEBIT entries. Below is a table that represents account types and how transaction entries affect their balances.

Account Type	Increase With	Decrease With
ASSET	DEBIT	CREDIT
LIABILITY	CREDIT	DEBIT
REVENUE	CREDIT	DEBIT
EXPENSE	DEBIT	CREDIT

Balance must be calculated dynamically from transaction entries.

You may:

- Compute in Python
- Or compute using SQL aggregation

Do not store balance as a mutable column.

2.3 Validation Requirements

You must enforce:

1. Account names are unique
2. Account name cannot be empty
3. Valid account type enum
4. Transaction has ≥ 2 entries
5. Total debits == total credits
6. Referenced accounts exist
7. Amounts are positive decimals
8. Description cannot be empty

Return proper HTTP codes:

- 400 – Validation error
- 404 – Resource not found
- 409 – Unique constraint violation

---

3. Architecture Requirements

We expect clean architecture principles.

3.1 Separation of Concerns

Clear boundaries between:

- API layer (FastAPI routers)
- Application/service layer
- Domain models (if applicable)
- Persistence models (SQLAlchemy, Pydantic)
- Database access layer

Business logic must NOT live in the API layer.

3.2 Database Design

- Proper relational schema
- Foreign keys
- Unique constraints
- NOT NULL constraints
- Indexes where appropriate
- Migrations are required

Suggested tables:

- accounts
- transactions
- transaction_entries

3.3 Code Quality Expectations

- Correct type hints everywhere
- Proper error handling, no 500 errors on API endpoints
- No business logic inside route handlers
- Use Decimal for money (NOT float)
- Clear and reasonable naming
- Small, testable functions
- Readable and properly formatted code, following PEP-8 best practices

---

4. Testing Requirements

Testing is mandatory.

4.1 Minimum Coverage

Unit Tests

Test:

- Transaction validation
- Balance calculation logic
- Edge cases

Business logic must be testable without running FastAPI.

Integration Tests

Test:

- Account creation
- Valid transaction creation
- Rejection of unbalanced transaction
- Balance calculation after multiple transactions

Use:

- Test PostgreSQL database
- OR testcontainers
- OR temporary DB with transaction rollback

4.2 Key Scenarios

At minimum test:

- Creating accounts
- Creating valid transaction
- Rejecting invalid transaction
- Calculating balances correctly
- Handling non-existent account

---

5. Bonus Features (Optional)

Optional additions:

- Transaction filtering (date range, account)
- Pagination
- Multi-currency support
- Detailed OpenAPI descriptions

Focus on core first.

---

6. Deliverables

6.1 What to Submit

1. Public Git repository
2. README.md including:
    - Architecture explanation

    - Design decisions
    - Setup instructions
    - How to run tests
    - What you'd improve
3. Working migrations
4. All tests passing

6.2 Git Best Practices

- Atomic commits
- Meaningful commit messages
- Proper .gitignore
- No IDE junk files

---

7. Evaluation Criteria

Criterion	Weight
Architecture	30%
Functionality	25%
Code Quality	20%
Testing	15%
Database Design	5%
Documentation	5%

---

8. Setup Recommendations

Quick Start

You may scaffold with:

- FastAPI project template
- Poetry init
- docker-compose for PostgreSQL

9. What We Are Looking For

We care about:

- Correct modeling of financial logic
- Clear architectural thinking
- Clean separation of layers
- Professional-level Python skills
- Thoughtful and careful validation

Final thoughts

If you have questions about:

- Requirements clarification: Feel free to make reasonable assumptions and document them
- Technical issues: Try to resolve them independently, but document any blockers
- Scope questions: Use your judgment - we want to see how you make decisions

We look forward to reviewing your solution and discussing your approach. Take your time to write code you'd be proud to show to your team.

Good luck!